RPG IV Mise en œuvre

Pointeurs

Concept

- RPG IV permet la déclaration et la manipulation de pointeurs
- Pointeur
  - Variable contenant une adresse mémoire
    - Adresse d'une donnée
    - Adresse d'une procédure
  - Valeur entière sur 16 octets
- Les pointeurs sont indispensables pour :
  - La gestion dynamique de la mémoire (liste chainée, tableau dynamique …)
  - L'utilisation des API IBM
  - …

Pointeur de données

Usage

- Un pointeur de données contient l’adresse mémoire d’une donnée
- Pour exploiter la donnée pointée, il faut une variable qui permette d’accéder à la donnée
  - Une variable basée sur le pointeur : l’adresse de la variable est définie par le pointeur
  - Une variable basée sur un pointeur n’a pas de mémoire affectée, elle agit comme un masque de lecture sur un emplacement mémoire
  - En modifiant l’adresse stockée dans le pointeur, on déplace le masque de lecture dans la mémoire

Déclaration

- Pointeur
  - Type
    - De déclaration : S (variable simple)
    - De donnée : *
  - Valeur par défaut : *NULL
- Variable basée sur un pointeur
  - Type
    - De déclaration : S (variable simple) ou DS (Data Structure)
    - De donnée : au choix (y compris *)
  - Valeur par défaut : non autorisé

Codes opérations et BIFs

- Nécessaires à la manipulation des pointeurs

Code Opération	Description	Format libre
ALLOC(E) %alloc	Demande d'allocation de mémoire	Oui
REALLOC(E) %realloc	Réallocation de mémoire allouée par ALLOC	Oui
DEALLOC(E/N)	Libération de l'espace alloué par ALLOC	Oui
%addr	Obtenir l'adresse d'une variable	Oui
%str	Gestion des chaines à terminaison nulle (C)	Oui
+, -	Affectation d'une nouvelle adresse par ajout ou soustraction d'un décalage (en nombre d'octets)	Oui

Exemple – données

```
.....Dname++++++++++ETDsFrom+++To/L+++IDc.Keywords+++++++++++++++++++Comments++++++++++

     D Ptr              s                *   inz( *NULL )
     D Variable         s              10a   inz( 'abcdefghij' )

     D Ptr2             s                *   inz( %addr( Variable ) )
     D Variable2        s               2a   based( Ptr2 )

     D Variable3        s                    like( Variable )
     D                                       based( Ptr3 )


       Ptr = %addr( Variable ) ;
       dsply ( 'Variable  : ' + Variable ) ;
       dsply ( 'Variable2 : ' + Variable2 ) ;

       Variable = 'qrstuvwxyz' ;
       dsply ( 'Variable  : ' + Variable ) ;
       dsply ( 'Variable2 : ' + Variable2 ) ;

       Ptr2 += %size( Variable2 ) ;
       dsply ( 'Variable  : ' + Variable ) ;
       dsply ( 'Variable2 : ' + Variable2 ) ;

       Ptr3 = Ptr ;
       dsply ( 'Variable3 : ' + Variable3 ) ;
```

```
abcdefghij
ab
```

```
qrstuvwxyz
qr
```

```
qrstuvwxyz
st
```

```
qrstuvwxyz
```

- Déclaration de pointeur
- Déclaration variable simple
- Déclaration variable pointée et pointeur
- La déclaration du pointeur peut être implicite
- Affectation d'une adresse à un pointeur
- Modification de la valeur pointée
- Déplacement du pointeur : avancement de la taille de Variable2
- Affectation de pointeurs

Exemple – mémoire

```
.....Dname++++++++++ETDsFrom+++To/L+++IDc.Keywords+++++++++++++++++++Comments++++++++++

      // Objectif : stocker en mémoire un nombre non déterminé
      // d'occurrences d'une valeur = tableau dynamique

     D Ptr              s                *   inz( *NULL )

     D Elem             s             300a   inz
     D NbElem           s               5u 0 inz

     D PtrW             s                *
     D ElemW            s                     based( PtrW )
     D                                        like( Elem )
     D CptW             s               5u 0 inz

       NbElem = 125 ;


       Ptr = %alloc( %size( Elem ) * NbElem ) ;

       PtrW = Ptr ;
       for CptW = 1 to NbElem ;
          ElemW = 'Valeur de l''élément n° ' + %char( CptW ) ;
          PtrW += %size( Elem ) ;
       endfor ;

       dealloc Ptr ;
```

Pointeur recevant l'adresse de l'espace alloué

Déclaration d'un élément et du nombre d'éléments à stocker

Variables de travail pour parcours de la mémoire

Nombre d'éléments à stocker

Allocation de l'espace mémoire

Parcours et alimentation de la mémoire

Libération de l'espace mémoire

Pointeur de procédure

Usage

- Un pointeur de procédure contient l'adresse de procédure
- Pour exploiter la procédure pointée, il faut un prototype de procédure
  - Cela permet d'appeler une procédure sans connaître à l'avance (à la compilation) son nom ou son emplacement
  - La procédure pointée doit respecter le prototype défini

Déclaration

- Pointeur
  - Type

    De déclaration : S (variable simple)

    De donnée : *

    Mot-clé : PROCPTR
  - Valeur par défaut : *NULL
- Procédure basée sur un pointeur
  - Type

    De déclaration : PR (PRocédure)

    Mot-clé : EXTPROC(pointeur)

Codes opérations et BIFs

- Nécessaires à la manipulation des pointeurs de procédure

Code Opération	Description	Format libre
%paddr	Obtenir l'adresse d'une procédure	Oui
CALLB	Appeler une procédure depuis son adresse	Oui
CALLP	Appeler une procédure depuis son adresse	Oui

Exemple – procédure (1/2)

```
.....Dname++++++++++ETDsFrom+++To/L+++IDc.Keywords+++++++++++++++++++Comments++++++++++

     dProc1            pr

     dProc2            pr                    extproc( 'Procedure2' )

     dProc3            pr                    extproc( PProc3 )
     dPProc3           s                 *   procptr

     dPProc            pr                    extproc( PPtr )

     d PPtr            s                 *   procptr
     d                                       inz( *null )

       // Affectation
       PPtr = %paddr( 'Proc1' ) ;
       PPtr = %paddr( 'Proc2' ) ;

       PPtr = PPRoc3 ;

       // appel :
       PPtr = %paddr( 'Proc1' ) ;
       PProc() ;   // appel Proc1

       PPtr = %paddr( 'Proc0' ) ;
       PProc() ;   // appel Proc2
```

Prototype Proc1

Prototype Proc2

Prototype Proc3
PProc3 est un pointeur

PProc est un prototype défini sur le pointeur PPtr

PPtr est un pointeur

Affectations possibles

La procédure réellement appelée est celle dont l'adresse est contenue dans PPtr

Exemple – procédure (2/2)

```
.....Dname++++++++++ETDsFrom+++To/L+++IDc.Keywords+++++++++++++++++++Comments++++++++++

     // Exemple « d'automate » : les procédures à utiliser
     // sont toutes inconnues au moment de la compilation

    dProcAAppeler     pr              n   extproc( PProcAAppeler )
    d  Data                        500a   const
    dPProcAAppeler    s               *   procptr inz

    dRetrouverProc    pr              *   procptr
    d  Fonction                     10a   value

    dDataW            s            500a   inz( 'Données' )

    dCst_Init         c                   const( 'INIT' )
    dCst_Calcul       c                   const( 'CALC' )

      // Appel procédure d'initialisation
      PProcAAppeler = RetrouverProc( Cst_Init ) ;
      if not ProcAAppeler( DataW ) ;
         // traiter retour
      endif ;

      // Appel procédure de calcul
      PProcAAppeler = RetrouverProc( Cst_Calcul ) ;
      if not ProcAAppeler( DataW ) ;
         // traiter retour
      endif ;
```

Prototype pour appels des procédures.
Pointeur de procédure.

Procédure retournant le pointeur sur la procédure à appeler

Retrouver le pointeur de la procédure à appeler et appel de la procédure

Usage

- L’usage des pointeurs nécessite de prendre en compte des difficultés
  - Lors du développement
    - Aucun contrôle à la compilation n’est possible
  - Lors de la mise au point
    - Les données sont plus difficilement accessibles
- Une attention particulière sur les tests
  - Il est possible de lire ou d’écraser n’importe quelle partie de la mémoire affectée au job !
  - Bug du « pointeur fou »